

Class Activity 15-07-26

Scenario 1: Smart City Air Quality Monitoring using r program

Code:

```
# Install package (Run only once)
```

```
install.packages("ggplot2")
```

```
# Load package
```

```
library(ggplot2)
```

```
# Create Sample Dataset
```

```
air <- data.frame(
```

```
  Location = paste("L", 1:15, sep=""),
```

```
  PM25 = c(25,40,55,70,35,90,65,80,50,45,30,95,60,75,85),
```

```
  NO2 = c(20,30,45,60,25,70,55,65,40,35,22,75,50,58,68),
```

```
  CO = c(0.5,0.8,1.2,1.5,0.6,2.0,1.6,1.8,1.0,0.9,0.7,2.2,1.4,1.7,1.9),
```

```
  Temperature = c(28,29,30,31,27,32,33,34,30,29,28,35,31,32,33),
```

```
  Humidity = c(60,58,55,52,65,50,48,45,57,59,62,43,51,49,46)
```

```
)
```

```
# Sequential Color Palette
```

```
ggplot(air, aes(x = NO2, y = PM25, color = PM25)) +
```

```
  geom_point(size = 5) +
```

```
  scale_color_gradient(low = "lightblue", high = "red") +
```

```
  labs(
```

```
    title = "Smart City Air Quality Monitoring (Sequential)",
```

```
    x = "NO2",
```

```
    y = "PM2.5",
```

```
    color = "PM2.5"
```

```
) +
```

```
theme_minimal()
```

```
# Diverging Color Palette
```

```
ggplot(air, aes(x = NO2, y = PM25, color = PM25)) +
```

```
  geom_point(size = 5) +
```

```
  scale_color_gradient2(
```

```
    low = "blue",
```

```
    mid = "yellow",
```

```
    high = "red",
```

```
    midpoint = 60
```

```
) +
```

```
labs(
```

```
  title = "Smart City Air Quality Monitoring (Diverging)",
```

```
  x = "NO2",
```

```
  y = "PM2.5"
```

```
) +
```

```
theme_minimal()
```

```
# Create Pollution Levels
```

```
air$Level <- ifelse(
```

```
  air$PM25 < 40,
```

```
  "Low",
```

```
  ifelse(air$PM25 < 70, "Medium", "High")
```

```
)
```

```
# Qualitative Color Palette
```

```
ggplot(air, aes(x = NO2, y = PM25, color = Level)) +
```

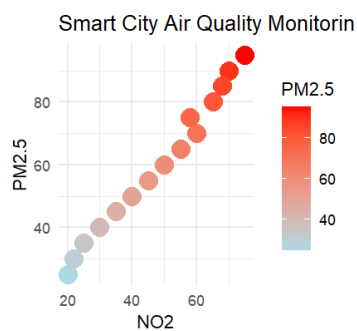
```
  geom_point(size = 5) +
```

```
labs(
  title = "Smart City Air Quality Monitoring (Qualitative)",
  x = "NO2",
  y = "PM2.5",
  color = "Pollution Level"
) +
  theme_minimal()
```

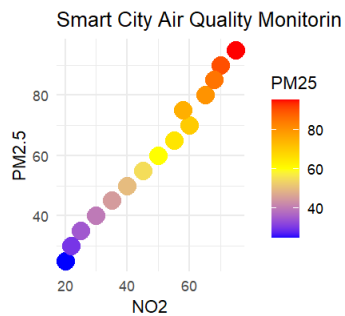
Output:

You will get **3 graphs**:

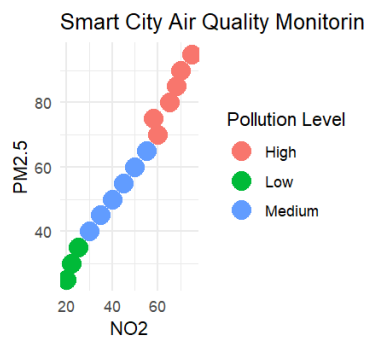
1. Sequential Color Plot



2. Diverging Color Plot



3. Qualitative Color Plot



Scenario 1: Smart City Air Quality Monitoring using python

Code

```
import matplotlib.pyplot as plt
```

```
# Sample Data
```

```
no2 = [20, 30, 40, 50, 60, 70]
```

```
pm25 = [25, 40, 55, 70, 85, 95]
```

```
# Sequential Color
```

```
plt.scatter(no2, pm25, c=pm25, cmap="Reds", s=120)
```

```
plt.title("Sequential Color Palette")
```

```
plt.xlabel("NO2")
```

```
plt.ylabel("PM2.5")
```

```
plt.colorbar(label="PM2.5")
```

```
plt.show()
```

```
# Diverging Color
```

```
plt.scatter(no2, pm25, c=pm25, cmap="coolwarm", s=120)
```

```
plt.title("Diverging Color Palette")
```

```
plt.xlabel("NO2")
```

```
plt.ylabel("PM2.5")
```

```
plt.colorbar(label="PM2.5")
```

```
plt.show()
```

```
# Qualitative Color
```

```
colors = ["green", "green", "orange", "orange", "red", "red"]
```

```
plt.scatter(no2, pm25, c=colors, s=120)
```

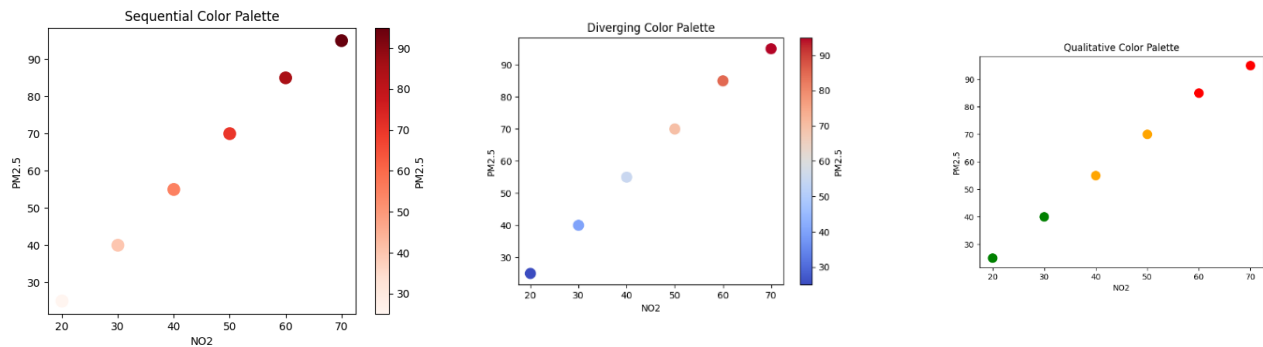
```
plt.title("Qualitative Color Palette")
```

```
plt.xlabel("NO2")
```

```
plt.ylabel("PM2.5")
```

```
plt.show()
```

Output



Scenario 2: Earthquake Risk Analysis using r program

Code:

```
earthquake <- data.frame(  
  Magnitude = c(1.2,2.1,3.0,4.5,5.2,6.1,7.3,8.0,9.1),  
  Depth = c(10,20,35,50,80,120,250,450,650)  
)
```

```
# Linear Scale
```

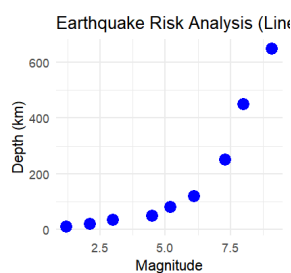
```
ggplot(earthquake, aes(x=Magnitude, y=Depth)) +  
  geom_point(color="blue", size=4) +  
  labs(  
    title="Earthquake Risk Analysis (Linear Scale)",  
    x="Magnitude",  
    y="Depth (km)"  
  ) +  
  theme_minimal()
```

```
# Log Scale

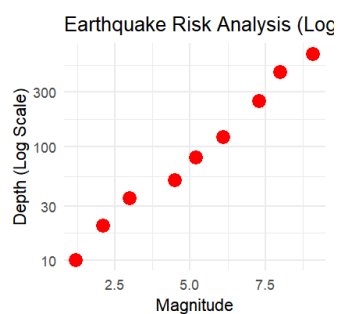
ggplot(earthquake, aes(x=Magnitude, y=Depth)) +
  geom_point(color="red", size=4) +
  scale_y_log10() +
  labs(
    title="Earthquake Risk Analysis (Log Scale)",
    x="Magnitude",
    y="Depth (Log Scale)"
  ) +
  theme_minimal()
```

Output

Graph 1: Linear Scale



Graph 2: Logarithmic Scale



Scenario 2: Earthquake Risk Analysis using python program

Code:

```
import matplotlib.pyplot as plt
```

```
magnitude = [1,2,3,4,5,6,7,8,9]
```

```
depth = [10,20,35,50,80,120,250,450,650]
```

```
# Linear Scale
```

```
plt.scatter(magnitude, depth, color="blue")
```

```
plt.title("Earthquake - Linear Scale")
```

```
plt.xlabel("Magnitude")
```

```
plt.ylabel("Depth")
```

```
plt.show()
```

```
# Log Scale
```

```
plt.scatter(magnitude, depth, color="red")
```

```
plt.yscale("log")
```

```
plt.title("Earthquake - Log Scale")
```

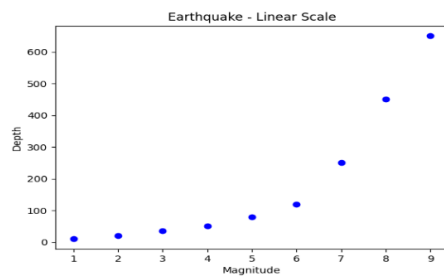
```
plt.xlabel("Magnitude")
```

```
plt.ylabel("Depth (Log)")
```

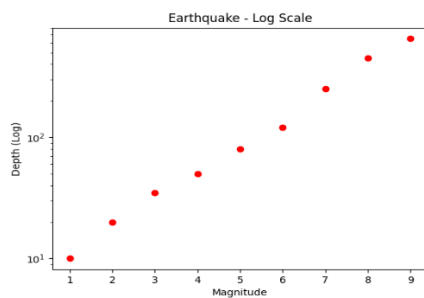
```
plt.show()
```

Output

- Linear Scale Plot



- Logarithmic Scale Plot



Scenario 3: Wind Direction Analysis using r program

Code:

```
wind <- data.frame(  
  Direction=c("N","NE","E","SE","S","SW","W","NW"),  
  Speed=c(15,20,10,25,30,18,12,22)  
)
```

Cartesian Plot

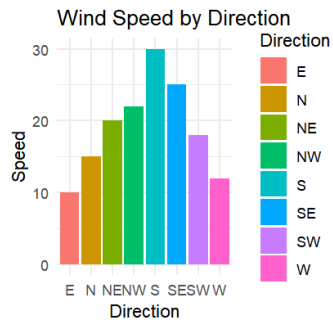
```
ggplot(wind, aes(x=Direction, y=Speed, fill=Direction)) +  
  geom_bar(stat="identity") +  
  labs(  
    title="Wind Speed by Direction",  
    x="Direction",  
    y="Speed"  
  ) +  
  theme_minimal()
```

Polar Plot

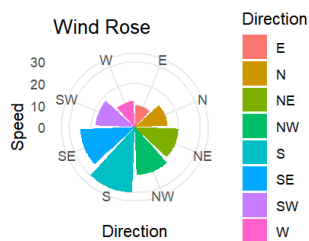
```
ggplot(wind, aes(x=Direction, y=Speed, fill=Direction)) +  
  geom_bar(stat="identity") +  
  coord_polar() +  
  labs(title="Wind Rose") +  
  theme_minimal()
```


Output

Bar Chart



Wind Rose (Polar Chart)



Scenario 3: Wind Direction Analysis using python program

Code:

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
direction = np.linspace(0, 2*np.pi, 8)
```

```
speed = [15,20,10,25,30,18,12,22]
```

```
# Cartesian Plot
```

```
plt.bar(range(8), speed)
```

```
plt.xticks(range(8), ["N","NE","E","SE","S","SW","W","NW"])
```

```
plt.title("Wind Speed")
```

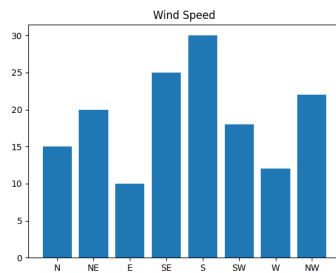
```
plt.show()
```

```
# Polar Plot
```

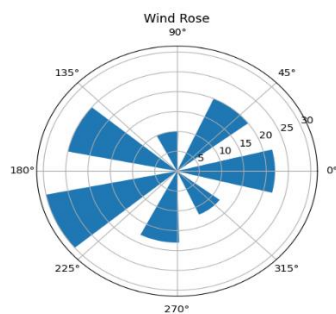
```
ax = plt.subplot(111, polar=True)
ax.bar(direction, speed, width=0.5)
plt.title("Wind Rose")
plt.show()
```

Output

- Bar Chart



- Polar Wind Rose



Scenario 4: Hospital Patient Monitoring Dashboard using r program

Code:

```
patient <- data.frame(
  Time=1:10,
  HeartRate=c(72,75,78,90,105,110,95,88,76,72)
)

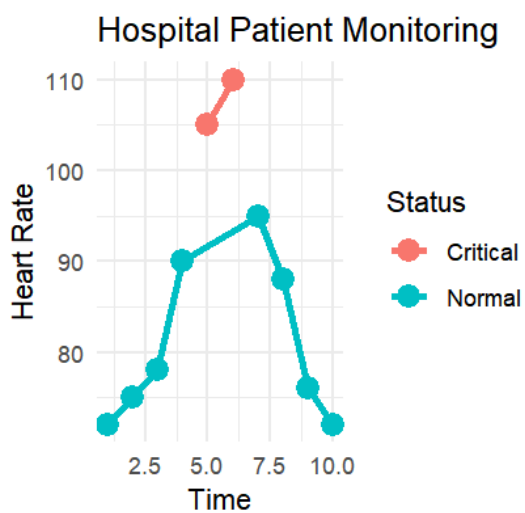
patient$Status <- ifelse(patient$HeartRate > 100,
  "Critical",
  "Normal")
```

```

ggplot(patient,
  aes(x=Time,
      y=HeartRate,
      color=Status)) +
  geom_line(size=1.2) +
  geom_point(size=4) +
  labs(
    title="Hospital Patient Monitoring",
    x="Time",
    y="Heart Rate"
  ) +
  theme_minimal()

```

Output



Scenario 4: Hospital Patient Monitoring Dashboard using python program

Code:

```
import matplotlib.pyplot as plt
```

```
time = [1,2,3,4,5,6,7,8,9,10]
```

```
heart_rate = [72,75,78,90,105,110,95,88,76,72]
```

```

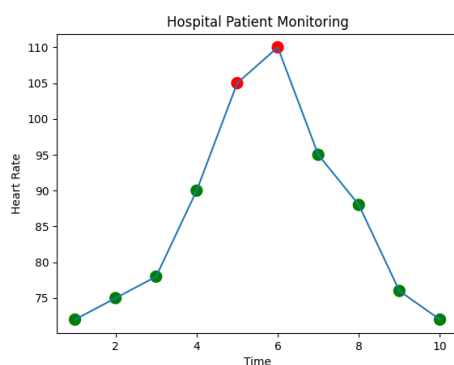
colors = []

for i in heart_rate:
    if i > 100:
        colors.append("red")
    else:
        colors.append("green")

plt.scatter(time, heart_rate, c=colors, s=100)
plt.plot(time, heart_rate)
plt.title("Hospital Patient Monitoring")
plt.xlabel("Time")
plt.ylabel("Heart Rate")
plt.show()

```

Output



Scenario 5: Global Climate Change Study using r program

Code:

```

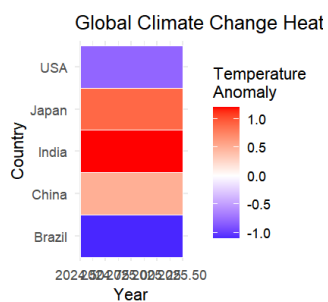
climate <- data.frame(
  Country=c("India","USA","China","Brazil","Japan"),
  Year=2025,

```

```
TempAnomaly=c(1.2,-0.8,0.5,-1.1,0.9)
)
```

```
ggplot(climate,
       aes(x=Year,
           y=Country,
           fill=TempAnomaly)) +
geom_tile(color="white") +
scale_fill_gradient2(
  low="blue",
  mid="white",
  high="red",
  midpoint=0
) +
labs(
  title="Global Climate Change Heatmap",
  fill="Temperature\nAnomaly"
) +
theme_minimal()
```

Output



Scenario 5: Global Climate Change Study using python program

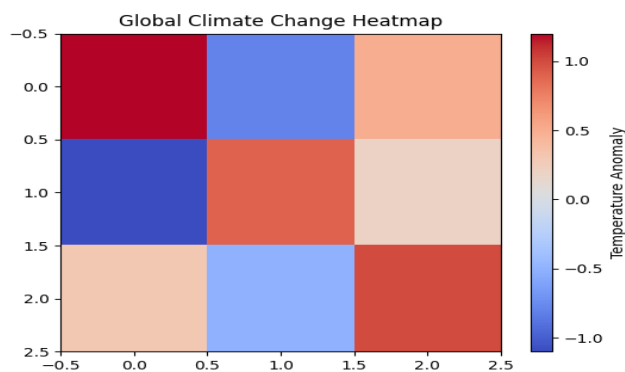
Code

```
import matplotlib.pyplot as plt
import numpy as np
```

```
data = np.array([
    [1.2, -0.8, 0.5],
    [-1.1, 0.9, 0.2],
    [0.3, -0.5, 1.0]
])

plt.imshow(data, cmap="coolwarm")
plt.colorbar(label="Temperature Anomaly")
plt.title("Global Climate Change Heatmap")
plt.show()
```

Output



Scenario 6: E-Commerce Sales Analytics using python program

Code:

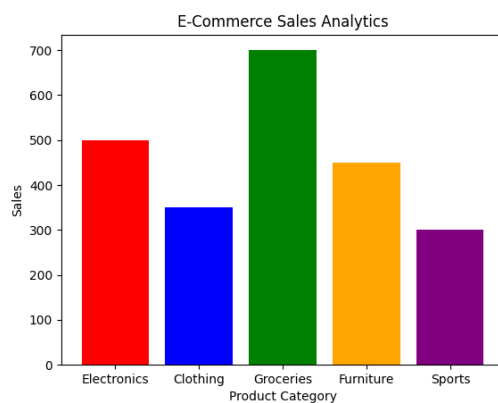
```
import matplotlib.pyplot as plt

categories = ["Electronics", "Clothing", "Groceries", "Furniture", "Sports"]
sales = [500, 350, 700, 450, 300]

plt.bar(categories, sales,
        color=["red", "blue", "green", "orange", "purple"])
```

```
plt.title("E-Commerce Sales Analytics")
plt.xlabel("Product Category")
plt.ylabel("Sales")
plt.show()
```

Output



Scenario 7: Satellite Image Vegetation Analysis using r program

Code:

```
vegetation <- data.frame(
  X = rep(1:5, each = 5),
  Y = rep(1:5, 5),
  NDVI = c(
    -0.8,-0.5,-0.2,0.1,0.3,
    -0.6,-0.3,0.0,0.4,0.7,
    -0.4,-0.1,0.2,0.6,0.8,
    -0.2,0.1,0.5,0.7,0.9,
    0.0,0.3,0.6,0.8,1.0
  )
)

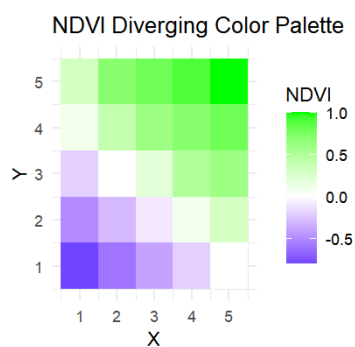
# Sequential Palette
ggplot(vegetation, aes(X, Y, fill = NDVI)) +
```

```
geom_tile() +
scale_fill_gradient(low = "yellow", high = "darkgreen") +
labs(
  title = "NDVI Sequential Color Palette",
  fill = "NDVI"
) +
theme_minimal()
```

Diverging Palette

```
ggplot(vegetation, aes(X, Y, fill = NDVI)) +
  geom_tile() +
  scale_fill_gradient2(
    low = "blue",
    mid = "white",
    high = "green",
    midpoint = 0
  ) +
  labs(
    title = "NDVI Diverging Color Palette",
    fill = "NDVI"
  ) +
  theme_minimal()
```

Output



Scenario 7: Satellite Image Vegetation Analysis using python program

Code:

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
ndvi = np.array([
    [-0.8,-0.5,-0.2,0.1,0.3],
    [-0.6,-0.3,0.0,0.4,0.7],
    [-0.4,-0.1,0.2,0.6,0.8],
    [-0.2,0.1,0.5,0.7,0.9],
    [0.0,0.3,0.6,0.8,1.0]
])
```

```
# Sequential Palette
```

```
plt.imshow(ndvi, cmap="Greens")
```

```
plt.colorbar(label="NDVI")
```

```
plt.title("Sequential Palette")
```

```
plt.show()
```

```
# Diverging Palette
```

```
plt.imshow(ndvi, cmap="RdYlGn")
```

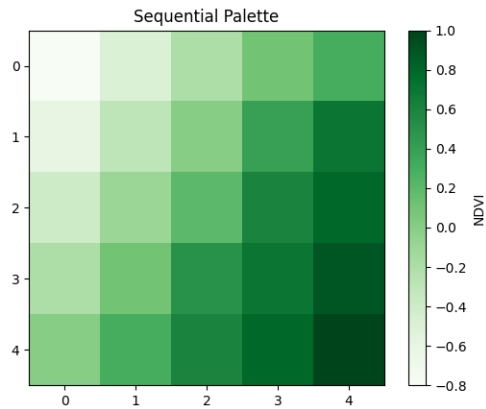
```
plt.colorbar(label="NDVI")
```

```
plt.title("Diverging Palette")
```

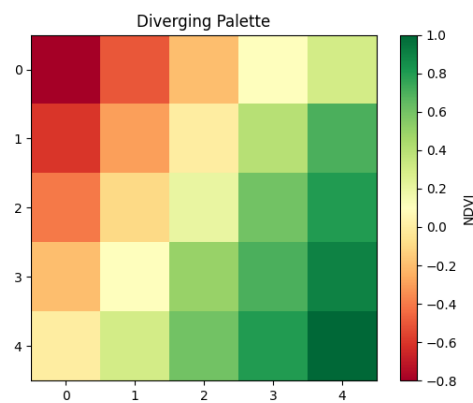
```
plt.show()
```

Output

- Heatmap using **Greens**



- Heatmap using **Red-Yellow-Green**



Scenario 8: Cryptocurrency Market Volatility using r program

Code:

```
crypto <- data.frame(
  Day = 1:10,
  Bitcoin = c(60000,62000,61000,63000,65000,67000,66000,68000,70000,72000),
  Ethereum = c(3000,3100,3050,3200,3300,3400,3350,3450,3550,3600)
)
```

```
crypto_long <- data.frame(
  Day = rep(crypto$Day, 2),
  Price = c(crypto$Bitcoin, crypto$Ethereum),
  Currency = rep(c("Bitcoin", "Ethereum"), each = 10)
```

)

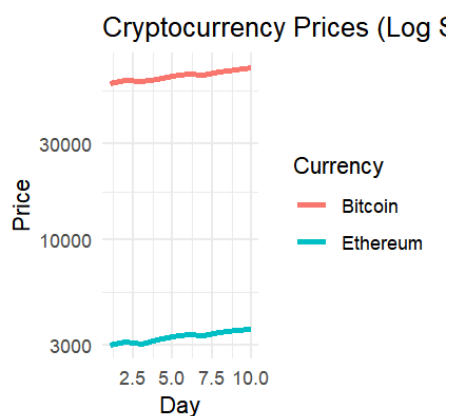
Cartesian Plot

```
ggplot(crypto_long,  
       aes(Day, Price, color = Currency)) +  
  geom_line(size = 1.2) +  
  labs(title = "Cryptocurrency Prices") +  
  theme_minimal()
```

Logarithmic Plot

```
ggplot(crypto_long,  
       aes(Day, Price, color = Currency)) +  
  geom_line(size = 1.2) +  
  scale_y_log10() +  
  labs(title = "Cryptocurrency Prices (Log Scale)") +  
  theme_minimal()
```

Output



Scenario 8: Cryptocurrency Market Volatility using python program

Code

```
import matplotlib.pyplot as plt
```

```
day = [1,2,3,4,5,6]
```

```
bitcoin = [60000,62000,61000,65000,68000,70000]
```

```
ethereum = [3000,3200,3100,3400,3500,3600]
```

```
# Normal Plot
```

```
plt.plot(day, bitcoin, label="Bitcoin")
```

```
plt.plot(day, ethereum, label="Ethereum")
```

```
plt.legend()
```

```
plt.title("Cryptocurrency Prices")
```

```
plt.show()
```

```
# Log Scale
```

```
plt.plot(day, bitcoin, label="Bitcoin")
```

```
plt.plot(day, ethereum, label="Ethereum")
```

```
plt.yscale("log")
```

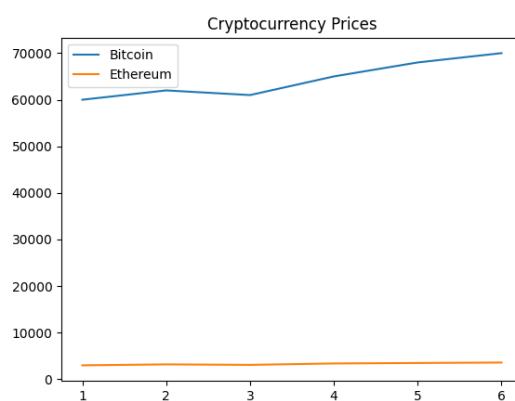
```
plt.legend()
```

```
plt.title("Cryptocurrency Prices (Log Scale)")
```

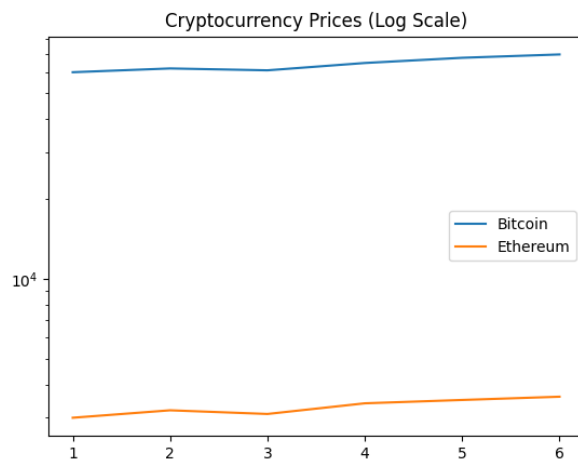
```
plt.show()
```

Output

- Normal Line Graph



- Logarithmic Line Graph



Scenario 9: Marathon Runner Performance Analysis using r program

Code:

```
runner <- data.frame(
  Direction = c("N", "NE", "E", "SE", "S", "SW", "W", "NW"),
  Speed = c(12, 14, 15, 16, 14, 13, 12, 15)
)
```

Cartesian Plot

```
ggplot(runner,
  aes(Direction, Speed,
    fill = Speed)) +
  geom_bar(stat = "identity") +
  labs(title = "Runner Speed by Direction") +
  theme_minimal()
```

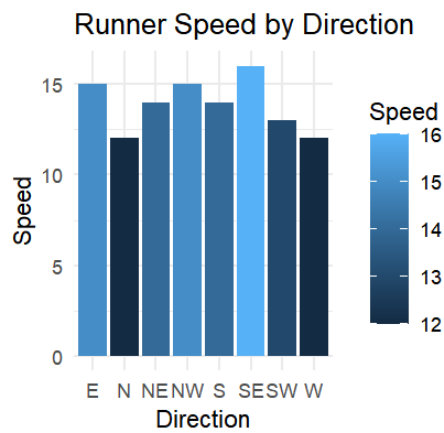
Polar Plot

```
ggplot(runner,
  aes(Direction, Speed,
    fill = Speed)) +
```

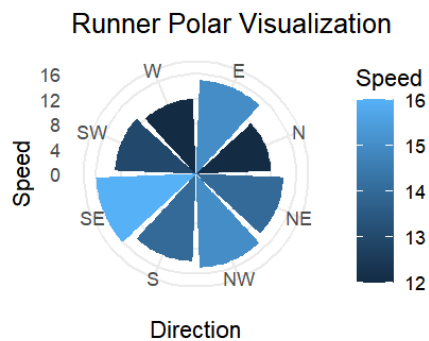
```
geom_bar(stat = "identity") +
coord_polar() +
labs(title = "Runner Polar Visualization") +
theme_minimal()
```

Output

- Cartesian Bar Chart



- Polar Coordinate Chart



Scenario 9: Marathon Runner Performance Analysis using python program

Code:

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
speed = [12,14,15,16,14,13,12,15]
```

```
direction = np.linspace(0,2*np.pi,8)
```

```
# Cartesian Plot
```

```
plt.plot(speed, marker="o")
```

```
plt.title("Runner Speed")
```

```
plt.xlabel("Direction")
```

```
plt.ylabel("Speed")
```

```
plt.show()
```

```
# Polar Plot
```

```
ax = plt.subplot(111, polar=True)
```

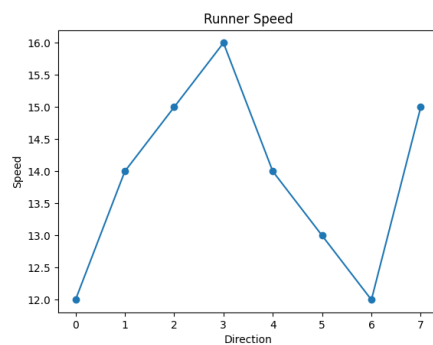
```
ax.scatter(direction, speed, c=speed, cmap="viridis", s=100)
```

```
plt.title("Runner Polar Plot")
```

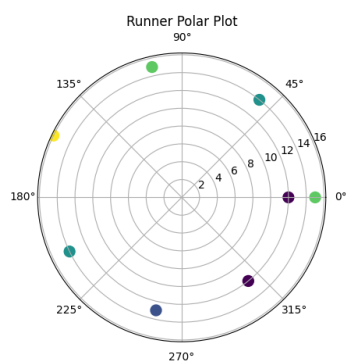
```
plt.show()
```

Output

- Line Chart



- Polar Scatter Plot



Scenario 10: COVID-19 Vaccination Campaign Dashboard using r program

Code:

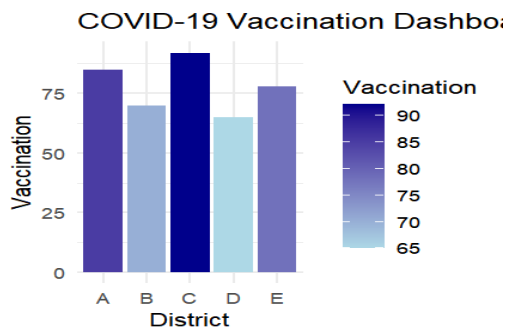
```
covid <- data.frame(
  District = c("A","B","C","D","E"),
  Vaccination = c(85,70,92,65,78)
)

# Blue Palette
ggplot(covid,
  aes(District,
    Vaccination,
    fill = Vaccination)) +
  geom_bar(stat = "identity") +
  scale_fill_gradient(low = "lightblue", high = "darkblue") +
  labs(title = "COVID-19 Vaccination Dashboard") +
  theme_minimal()

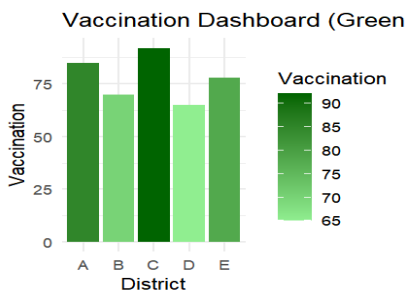
# Green Palette
ggplot(covid,
  aes(District,
    Vaccination,
    fill = Vaccination)) +
  geom_bar(stat = "identity") +
  scale_fill_gradient(low = "lightgreen", high = "darkgreen") +
  labs(title = "Vaccination Dashboard (Green Palette)") +
  theme_minimal()
```

Output

- Bar Chart using Blue color palette.



- Bar Chart using Green color palette.



Scenario 10: COVID-19 Vaccination Campaign Dashboard using python program

Code:

```
import matplotlib.pyplot as plt
```

```
district = ["A","B","C","D","E"]
```

```
vaccination = [85,70,92,65,78]
```

```
# Blue Palette
```

```
plt.bar(district, vaccination, color="skyblue")
```

```
plt.title("Vaccination Dashboard (Blue)")
```

```
plt.xlabel("District")
```

```
plt.ylabel("Vaccination Rate")
```

```
plt.show()
```

```
# Green Palette
```

```
plt.bar(district, vaccination, color="green")
```

```
plt.title("Vaccination Dashboard (Green)")
```

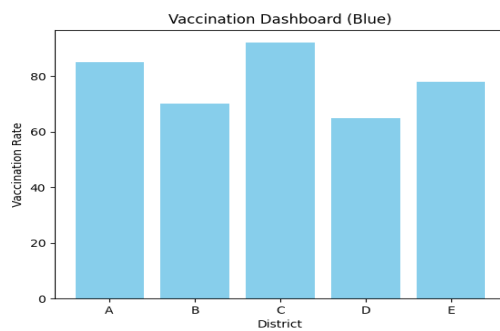
```
plt.xlabel("District")
```

```
plt.ylabel("Vaccination Rate")
```

```
plt.show()
```

Output

- Bar chart with **Blue** palette.



- Bar chart with **Green** palette.

